

Paper number: P1490r0
Date: 2018-02-13
Author: Andrzej Krzemieński
Email: akrzemi1 (at) gmail (dot) com
Audience: EWG (Evolution)

Contract-related issues

This paper attempts to summarize the issues with the current contract design that have come up in the recent discussions in the EWG reflector. The goal is to give the picture of the situation to all Committee members.

Convention in this paper

A *Contract Checking Statement* or a *CCS* is any attribute declaring a precondition, a postcondition, or an assertion, like `[[expects: cond]]`.

An *axiom-level CCS* is a CCS with level `axiom`, like: `[[expects axiom: cond]]`.

In the examples we only mention preconditions. This is because this is the preconditions where most of the miscommunication between two different parties is observed, and where the biggest benefit of contracts support is expected.

In this document we treat "contract-based assumptions" and "contract-based optimizations" as synonymous: if one can assume that a certain condition holds, one can use this assumption for optimization purposes (such as branch elimination).

1. Contract-based optimizations that cause concerns.

The problem that [\[P1290r1\]](#) intends to address is related to the following statement in the [\[WD\]](#) (`[dcl.attr.contract.check]/p4`):

[...] it is unspecified whether the predicate for a contract that is not checked under the current build level is evaluated; if the predicate of such a contract would evaluate to `false`, the behavior is undefined.

This unspecified behavior is referred to as "predicate can be assumed to be true". This works under the general mechanism of UB: compiler is allowed to assume that UB never happens in the program. This is the heart of UB-based optimizations. Because evaluating unchecked predicates is implementation-defined the implementation is allowed (but not required) to nonetheless perform a run-time check. Because in another place ([dcl.attr.contract.check]/p6) we require that any side effect while evaluating the precondition is UB, under the as-if rule, evaluating the precondition at run-time is equivalent to evaluating it symbolically (at compile-time), so the implementation is allowed to understand the condition. If it can convince itself (prove) that there is a path in the program that would cause the predicate to evaluate to false it would mean that this path leads directly to UB. Since compiler is allowed to assume that UB never happens, it can infer from this that this path is never taken, and from the latter it can infer that the predicate in the CCS is never false. then it can use this assumptions in other places, in particular, *before* the place where the predicate appears, which we often call "time travel" optimizations.

(Note that "time travel" optimizations are already implemented in compilers and standard-compliant, even without any contracts. The question here is only if we want to add more opportunities for these optimizations.)

One interpretation of the above [dcl.attr.contract.check]/p4 is this: compilers are allowed to implement this line of reasoning, so there is a risk that there may be even one compiler that will do it *without* giving the programmers the chance to turn this optimization off, or with these optimizations enabled by default. Therefore giving the compiler vendors even the *possibility* to implement this optimization is perceived as a risk. The risk is especially visible to library authors that target multiple compilers. Especially, if they retrofit CCSs to existing code, as explained later in the paper.

A different way of looking at this definition is that compiler vendors can communicate with programmers by other means than the International Standard. For instance, even though the International Standard does not demand that implementations give the programmers control to opt out of any optimizations, compilers nonetheless give them this option. Or, even though the International Standard allows optimizations based on strict type aliasing rules (UB-based optimizations) and does not mandate a "mode" where these optimizations (assumptions) would be disallowed, compilers like clang or gcc still allow the programmers to control this with flag `-fstrict-aliasing`.

This point of view is based on trust that compiler vendors work in the best interests of the programmers, that they are aware of the issues related to assuming the CCS predicates, and that their natural course of action would be to either never implement such assumptions, or enable them under a compiler flag: not enabled by default, possibly with additional options to selectiely choose which CCSs to use for assumptions. Under this view there is no problem related to contract-based optimizations in the current [WD] (except maybe for replacing unspecified behavior with implementation-defined behavior): it gives the compiler vendors a provision for implementing contract-based optimizations in a responsible way.

Thus, one of the factors that drives the answer to the decision whether to mandate no-contract-based-optimization guarantee in the Standard is whether we trust that compiler vendors will be responsible when doing their job.

2. The goal of having axiom-level CCSs

We have seen two positions here. One position is that an axiom-level CCS can be used to express a precondition to a function (something that we expect, but we are not sure if the callers will comply with) formally where the predicate must never be evaluated at run-time, because it is unimplementable, or would have side effects or has an unacceptable complexity even for test builds. A common example is `is_reachable()` for a pair of iterators that are expected to represent a range. This position is justified by the name "level" we use in the standardese: a level of difficulty with evaluating the condition:

- `default` -- no difficulty,
- `audit` -- may noticeably impact program performance,
- `axiom` -- just cannot be evaluated.

The other position, is that axiom-level CCSs, unlike other CCSs, do not express expectations (something we need but cannot be sure about), but declare program-wide "absolute truths" that compiler is allowed to take for granted: they enable UB-based optimizations and are not allowed to be checked. This position is backed by the fact that preprocessing token `axiom` spells the same as the notion of an axiom in mathematics. This position precludes defining preconditions like `is_reachable()` described above.

The difference between the two positions can be illustrated with the following example. We have a functon with axiom-level CCS:

```

template <InputIter It>
void algo(It b, It e)
    [[expects axiom: is_reachable(b, e)]];

```

Now, the caller of this function has the following body:

```

void caller(int * b, int * e) // concrete pointer types
{
    if (std::greater<int*>{}(b, e)) // sanity check
        std::cerr << "bad range provided \n";

    algo(b, e);
}

```

Expression `not std::greater<>{}(b, e)` is a runtime-checkable subset of a generally non-checkable `is_reachable(b, e)`. The latter implies the former.

If `is_reachable(b, e)` is true then condition `std::greater<>{}(b, e)` surely cannot be true. In the "absolute truths" position, compiler can take the former for granted, therefore it can eliminate the sanity check altogether (effectuating a time-travel optimization).

This also illustrates that optimizations based on axiom-level CCSs can introduce new paths in the program (potentially with UB) in the same manner as default and audit level. If optimizations based on default- and audit-level CCSs cause concern, then so should the axiom-level ones.

The current [\[WD\]](#) technically accommodates both positions but this is a superficial unification. Programmers and tool writers will need to be given instructions how to use axiom-level CCSs. And if they are given contradictory guidance this will create a fracture in the community even though the International Standard clearly defines what happens under each mode with axiom-level CCSs. This is because the use of CCSs is beyond what the IS can describe (like static analysis tools).

The consensus regarding the two positions needs to be reached even if it is not expressed directly in the IS. It can be used as a basis for applying fixes and future modifications to contracts. For instance, the solution in [\[P1290r1\]](#) takes the direction of the second position. It adds a new semantic distinction between axiom-level CCSs and other CCSs: the former are allowed to be used for UB-based optimizations under one mode, whereas other levels of CCSs are not allowed for UB-based optimizations under any defined mode. This

distinction has no justification (based on the conceptual model) if we assume the first position (where axiom-level CCSs differ only in the guarantee that they will never be run-time evaluated). But it makes perfect sense if we assume the second position.

Therefore the decision which position we adopt should be made prior to considering [\[P1290r1\]](#). This is in order for a high level design to control the details and not vice versa.

3. How do we define "assume" for axiom-level CCSs?

There is a smaller problem related to assumptions (optimizations) based on axiom-level CCSs. In the [\[WD\]](#) as well as in [\[P1290r1\]](#) it is defined as:

[...] it is unspecified whether the predicate for a contract that is not checked under the current build level is evaluated; if the predicate of such a contract would evaluate to `false`, the behavior is undefined.

This means that an implementation is allowed at its discretion to evaluate a condition inside an axiom-level CCS. This means that if my condition has side effects, and the implementation chooses to evaluate my condition, even though I used the `axiom` level, I can still get UB even if condition returns true.

This definition works for `default` and `audit` levels, because here it is obvious that conditions should not have side effects. However, for level `axiom` one use case that was mentioned in the reflectors is when a predicate is "generally reasonable" but has side effects, such as:

```
template <InputIter It>
void read_first_three(It b, It e)
    [[expects axiom: std::distance(b, e) >= 3]];
```

We should agree whether such use case is intended and should be supported. If so, the definition of what it means for such axiom-level CCS condition to be assumed needs to be changed.

[\[P1429r0\]](#) tries to address this by providing an alternative definition of an assumption:

A contract having the semantic *assume* is not checked and not evaluated. It is undefined behavior if the predicate of such a contract would evaluate

to false.

However, while intuitively correct, the construct "would evaluate to false" seems not to be well defined.

4. Can we put functions without definitions in axiom-level CCSs?

The current [\[WD\]](#) says that conditions in axiom-level CCSs are as any other conditions: they are ODR-used and, as per the provision in [\[dcl.attr.contract.check\]/p4](#), the implementation is allowed to invoke them at run-time. Therefore, it is not clear if not providing a definition is a linker error or not.

Requiring the function body to exist has some applications: for some predicates we know how to express them, but we do not want them to be evaluated (too expensive, have side effects), but static analysis tools may wish to instantiate the bodies to observe the code and draw conclusions.

On the other hand this disallows declaring (but not defining) predicates whose body just cannot be implemented, like `is_reachable()`.

Whichever is chosen, it may compromise the other's use cases. The design document for contracts, [\[P0380r0\]](#), seems to have intended the second (not ODR-used) approach:

Note that the predicate of an axiom is not evaluated and may contain calls to undefined functions.

5. Fewer use cases for continuation mode

The design document, [\[P0380r0\]](#), gave two use cases for continuation mode:

There are programs that need to resume computation after executing a violations handler. This seems counterintuitive ("continue after a precondition violation!!!?"), but there are two important use cases:

- Gradual introduction of contracts: Experience shows that once you start introducing contracts into a large old code base, violations are found in "correctly working code." In other words, contracts are violated in ways that did not cause crashes for the actual use of the code for the actual data used. There are examples where the number of such "currently harmless violations" is massive. The way to cope is

to install a violation handler that logs the problem and continues. This allows gradual adoption.

- Test harnesses: Contracts are code, so they can contain bugs. To test contracts, we need to execute examples of violations. A convenient way of organizing such a test suite is to have a violation handler throw an exception and have the main testing loop catch exceptions and proceed running the next test

Regarding the second use case, during the course of discussion and clarifications, it has been made clear that one does not need the continuation mode to safely throw from a violation handler and continue the program execution. `std::terminate()` is only called when the violation handler returns normally.

The first use case requires more elaboration.

Retrofitting preconditions

Adding a precondition to an existing function is tricky. It could introduce UB in programs that previously did not have any. Compare the following function:

```
int f(int * p)
// expects: p != nullptr
{
    if (p == nullptr) // safety check
        throw std::logic_error{""};
    return *p;
}
```

The function has a narrow contract, but it was advertized informally, so "just in case" it makes an internal check. It is possible that in the program this function is called out of contract, but the outcome is tolerable: some part of the operation is cancelled but the rest works fine. For instance, the operation works as the user expects but it is not logged as other operations. Users are told, "be careful with this button, but otherwise the application is fine." But if we add the formal precondition things may get changed to worse:

1. If contract-based optimizations are enabled, compiler can assume that `p` is never null and can completely elide the safety check inside function body allowing the null pointer to be dereferenced which was not the case when the CCS was not there.

2. If CCSs are runtime-checked and terminate the program on violation failure, we get a crash in place of a benign bug.

[\[P1290r1\]](#) addresses the first problem by disallowing assumptions in CCSs altogether under any circumstances. (Except for axiom-level CCSs.)

Continuation mode is supposed to address the second problem. If there is no `std::terminate()` called, and the violation handler only logs the violation event, the program can move on, as before.

Problems with continuation mode

However, global continuation mode creates new problems. One problem is that in one TU we might already have "secured" runtime-checked preconditions that are running in production, and `std::terminate()` on violation (and this is ok, because these violations occur rarely enough) and protect the program from UB, e.g.:

```
int f(int * p)
  [[expects: p != nullptr]]
{
  return *p;
}
```

Now, we want to retrofit some preconditions in other functions, but in order not to `std::terminate()` on the new benign precondition violations we need to turn the continuation mode. The moment we do it, the precondition in function `f` no longer protects the function body against the null pointer dereference. Because of that the compiler is allowed to assume that `p` is never null and propagate this assumption even back in time, to the callers of function `f`, also eliding other runtime CCS checks that test for `p` being null.

Another example of an UB introduced by continuation mode. Imagine a function with two precondition CCSs, they are "secured": runtime-checked and call `std::terminate()`.

```
int f(int * p)
  [[expects: p != nullptr]]
  [[expects: *p >= 0]];
```

If default CCSs are runtime-checked and the pointer happens to be null, the runtime check in the first CCS will cause the program to log the violation and

terminate. But now we turn the continuation mode on, in order to retrofit some new CCSs elsewhere in the TU. This affects our function `f` with secured preconditions. Now, after reporting the violation in the first CCS we will proceed to evaluating the second and dereference the null pointer. But the program will not crash! If we compile with optimizations enabled (which includes UB-based optimizations) the compiler can assume that UB never happens, therefore it can assume that `p` is never null, therefore it will elide the first CCS entirely. Even if runtime checking is on. This is not due to contract-based optimizations: this is due to normal UB-based optimizations.

Thus, the continuation mode handles its use case by introducing potential UB and surprising results.

[P1429r0] addresses the problem of the "gradual introduction of contracts" by offering a per-CCS control whether the CCS is a retrofit or a "baked" condition. Only after the "retrofit" CCS condition violation does the control continue. For the remaining CCSs the violation means `std::terminate()`. It also addresses the problems with optimizations based on retrofitted CCSs: they have fixed semantics: either "ignore" or "check and continue": there is no chance of enabling contract-based optimizations on such CCS even if contract-based optimizations are enabled for all CCS levels.

This is a design change (at the last minute), but admittedly one that addresses the design goal of [P0380r0]: gradual introduction of contracts. It seems a superior solution to continuation mode.

The questions to be answered here are:

1. Do we want C++20 to address the use case of the gradual introduction of contracts, given that the solution with continuation mode causes problems of its own?
2. If we want to address the use case, can we afford the last minute changes?

Acknowledgements

The contents of the paper were based on the input from many discussion participants, including John Lakos, Joshua Berne, Ville Voutilainen, J. Daniel Garcia, Bjarne Stroustrup, Nathan Myers, Peter Dimov, Mathias Stearn, and many more.

Tomasz Kamiński has reviewed the document and suggested numerous improvements.

References

[\[WD\]](#) -- Richard Smith, N4800, "Working Draft, Standard for Programming Language C++".

[\[P0380r0\]](#) -- G. Dos Reis, J. D. Garcia, J. Lakos, A. Meredith, N. Myers, B. Stroustrup, "A Contract Design".

[\[P1290r1\]](#) -- J. Daniel Garcia, Ville Voutilainen, "Avoiding undefined behavior in contracts".

[\[P1429r0\]](#) -- Joshua Berne, John Lakos, "Contracts That Work".